OMNITRIX
CAREER WORKSPACE

CA-3 / DOMAIN-SPECIFIC AGENTIC AI PoC

OMNITRIX

PROJECT REPORT

Career opportunity discovery, validation and application tracking

BY

Aadya Rastogi
24070126002

Atherva Srivastava
24070126043

AIML A

SYMBIOSIS
INSTITUTE OF TECHNOLOG



Executive Summary

Omnitrix is a Streamlit proof of concept for the student career-operations domain. It helps a student search a bundled opportunity dataset, compare roles with a profile, review listing quality, and keep application dates and notes in one tracker. Its purpose is to make a scattered manual process easier to inspect and manage; the project does not claim measured time savings or live vacancy coverage.

The system demonstrates agentic behavior through an explicit, ordered workflow implemented in Python. A Planner turns a request into a task plan. Discovery filters local records, Matching computes an explainable fit score, Follow-up proposes date-based actions, and a Critic validates listings and suggested actions. A Tracker writes to SQLite only when the student chooses a save or update action. The Agent Workflow page exposes the plan and agent log.

The PoC is deterministic and does not use LangChain, LangGraph, MCP, a hosted model, or a paid API. The dataset contains 30 synthetic listings. Results are therefore suitable for demonstrating orchestration and validation, not for making real job-search decisions.

Project at a Glance

Item	Implementation
Domain	Education and career operations
Problem	Scattered internship discovery, duplicate or incomplete listings, missed dates, and fragmented application tracking
Interface	Streamlit app with Dashboard, Find Opportunities, My Applications, Deadlines & Follow-ups, Resume Tailor, and Agent Workflow pages
Workflow	Sequential Python function calls coordinated by opportunityos/orchestrator.py
Data	30 bundled synthetic opportunity records; local SQLite application tracker
Controls	Human approval for tracker changes and resume confirmations; validation before results are presented

GITHUB LINK: https://github.com/ATS-27/omnitrix

1. Problem and Domain

1.1 Domain justification

The project focuses on education and career operations because students often need to compare entry-level roles across job boards while working with a limited time window and incomplete information. Finding a promising role is only one part of the task: students also need to assess fit, notice closing dates, avoid duplicate posts, and remember what they have already done.

1.2 Existing process and challenge

In a typical manual process, a student searches several platforms, opens posts individually, compares requirements with their skills, copies dates into a spreadsheet or notes app, and checks each application later. This creates repeated work and makes errors easy: the same role may appear more than once, a deadline may be missing or invalid, and a follow-up can be forgotten. Omnitrix models these pain points with synthetic records so that the workflow can be demonstrated safely without representing the records as real openings.

1.3 Problem statement

Students lack one transparent workflow for finding relevant internship opportunities, checking basic listing quality, and tracking their own application next steps. The PoC demonstrates how a coordinated set of software agents can organize those tasks while keeping the student in control of application and resume changes.

2. Proposed Solution and Scope

Omnitrix accepts a natural-language search request and a student profile. It uses the request to select a workflow and infer a small set of filters, searches the local dataset, scores profile fit, validates the records, and presents a ranked result with explanations. The user may save a result to the application tracker. Recorded dates can inform follow-up suggestions, and the separate Resume Tailor workflow maps job requirements to resume evidence and prepares reviewable changes.

Included in the PoC

- Profile-based search across the bundled synthetic CSV, with optional type, work-mode, location, role, skill, and deadline filters.
- Explainable fit scores based on documented weights and the fields in the profile and listing.
- Critic checks for required fields, date formats, closed listings, duplicates, missing deadlines, unclear status, and source availability.
- A SQLite tracker for user-saved roles, statuses, application dates, follow-up dates, notes, and next actions.
- Suggested next steps based on recorded dates, labelled as suggestions rather than verified employer updates.
- Resume evidence mapping, clarification questions, reviewable edits, truthfulness checks, and gated export.

Outside the current scope

- Live job-board search, employer verification, automatic application submission, and notifications.
- Predictive or learned ranking; current scoring is a fixed heuristic based on available fields.
- Separate autonomous LLM agents or communication between model endpoints.

3. Agent Architecture and Communication

The agents are Python components called in sequence by run_workflow() in opportunityos/orchestrator.py. They exchange ordinary Python dictionaries, lists, and dataclasses through function arguments and return values. A shared WorkflowLog collects readable entries for the UI. This is a lightweight orchestration design: no agent sends natural-language messages to another model, and no agent runs as a separate service.

3.1 Workflow diagram

Stage	Input	Action and hand-off	Output
Student and UI	Search text, profile, optional filters	Collects the request and lets explicit form filters override inferred filters	Request context
Planner	Request text	Routes to find, deadlines, or follow-ups; sets ordered steps and keyword-inferred filters	Plan dataclass
Discovery	Plan filters and local CSV records	Filters by listing type, mode, location, role, skill, and/or date window	Candidate records
Matching	Candidates and student profile	Computes weighted score, matched/missing skills, and explanation	Scored records
Follow-up	Tracker rows, scored records, as-of date	Creates suggested actions from recorded application or listing dates	Action records
Critic	Scored records and actions	Checks fields, dates, duplicate groups, source and status caveats, explanation consistency, and action evidence	Validated, excluded, warned, and approved suggestions
UI and Tracker	Validated result and user choice	Displays results; Tracker writes only after an explicit save or edit	Streamlit view and SQLite record

The deadline workflow skips ordinary profile ranking and uses a lighter validation path. The follow-up workflow can review tracker records without running listing discovery. This lets the Planner select only the steps relevant to the request.

3.2 Agent responsibilities

Agent	Responsibility	Current method
Planner	Choose the task and ordered workflow steps	Transparent keyword rules; deterministic and testable
Discovery	Retrieve possible records	Local in-memory filtering; no network calls
Matching	Rank records and explain fit	Weighted heuristic using profile/listing fields
Follow-up	Suggest next steps	Rules based on recorded dates and tracker status
Critic	Validate and flag outputs	Field, date, status, duplicate, score explanation, and action-basis checks
Tracker	Persist user-approved changes	SQLite operations behind explicit UI actions
Resume workflow	Map requirements to evidence and guard edits	Parser, gap analysis, clarification, tailoring, and truthfulness checks

4. Ranking and Validation

4.1 Explainable fit score

Each listing receives a 0–100 score. The score adds points for profile overlap rather than inferring qualifications beyond the supplied profile and listing. Skill overlap accounts for 50 points, preferred role for 20, location and work mode together for 15, opportunity type for 10, and deadline fit for 5. The configured weights are fixed for this PoC and sum to 100. The UI also shows matched and missing skills and a textual explanation assembled from the same fields.

Scoring dimension	Weight	Evidence used
Skills	50	Share of listed required skills also present in the profile
Role	20	Preferred role phrase or shared words in the role title
Location and work mode	15	Whether listing location and mode match profile preferences
Opportunity type	10	Whether internship, research, or other type matches preference
Deadline	5	Whether a valid deadline is available and how far away it is

4.2 Critic checks and outcome labels

The Critic excludes records that are missing required fields, have malformed dates, are marked closed, duplicate a higher-ranked record, or have an explanation that does not match the profile/listing facts. It flags rather than hides uncertainty where possible: unknown deadlines, expired dates, unclear listing status, missing source URLs, and records not marked synthetic receive visible warnings. The bundled records carry a synthetic label. The app does not establish that an employer has posted or updated a role.

Follow-up suggestions must include a due date and a recorded-date basis to pass the action validator. The app presents them as suggestions and does not claim to know an employer's status.

5. Application Workflow and User Interface

Page	Purpose and interaction
Dashboard	Summarizes tracked roles, statuses, priority actions, recent applications, and profile-based recommendations when available.
Find Opportunities	Submits a search request and optional filters; shows workflow counts, ranked records, score rationale, warnings, and save actions.
My Applications	Edits status, applied/follow-up dates, notes, and next action; permits removal from the tracker.
Deadlines & Follow-ups	Shows suggestions derived from recorded dates and validated listings closing soon.
Resume Tailor	Accepts a resume or sample, compares it with a job description, asks about evidence, and gates downloads on truthfulness checks.
Agent Workflow	Shows the plan, ordered steps, agent hand-offs, counts, warnings, exclusions, and workflow log after a search.

The sidebar profile, preferred role/location/work-mode/type settings, as-of date control, scoring details, and demo tracker tools support the main workflow. Streamlit session state retains the selected page and workflow/resume results across reruns. Search and resume analysis use forms so the main processing occurs on submission rather than on each field edit. Listing and resume analysis helpers use Streamlit caching.

6. Data, State, and Human Oversight

The listing source is data/listings.csv, containing 30 synthetic records. The examples intentionally include edge cases such as repeated posts, a missing deadline, malformed date text, a closed listing, and a missing source link. This allows the validation path to be demonstrated without presenting fictional vacancies as live opportunities. Sample resume and job-description files are fictional as well.

Application records are stored locally in data/tracker.db using SQLite. The Tracker agent is the only component that writes application changes, and it is called only from explicit save, update, or remove controls. Search results and recommendations do not submit applications. Resume export creates a draft in a temporary directory; the master resume is not edited in place.

The system uses a fixed demonstration date for repeatable runs, with a sidebar option to use today's date. This makes deadline examples reproducible while allowing an alternate date context during a demonstration.

7. Demonstration Plan and Expected Evidence

1. On Dashboard, state the search and tracking problem and clarify that the bundled listings are synthetic.
2. Open Find Opportunities and run the prefilled AI/NLP internship request. Point to retrieved, validated, excluded, and suggested-action counts.
3. Open a result's score explanation and describe which profile and listing fields contributed to the score.
4. Open Agent Workflow to show the task plan, sequence, agent log, validation decisions, warnings, and excluded-record reasons.
5. Save a role and show that it appears in My Applications. Update a status or date, then view date-based suggestions in Deadlines & Follow-ups.
6. Open Resume Tailor with the sample materials. Review evidence gaps and a proposed change, then show that unsupported claims block export.

Expected evidence is visible in the application: deterministic counts and log entries, ranked results with explanations, Critic exclusions/warnings, and tracker rows after user action. Performance gains are not measured in this PoC; any claim that the workflow saves time should be evaluated with users and a defined comparison task.

8. Limitations and Future Work

Current limitations

- The dataset is static, synthetic, and local. Source URLs are placeholders; the app does not fetch or verify live postings.
- The Planner uses keyword rules, and Matching uses a fixed heuristic. These methods are transparent but may miss context and synonyms.
- Deadline and status information is only as reliable as the stored record. A warning is not equivalent to employer verification.
- The resume parser and truthfulness checks are guardrails, not a guarantee that every claim is accurate or every document format is handled perfectly.
- The workflow runs in one Python process with sequential calls; it does not provide distributed execution, model-based planning, or inter-agent messaging.

Practical next steps

- Connect an approved opportunity source with terms-compliant access, attribution, timestamps, and human review before displaying records as current.
- Evaluate ranking with student feedback and labeled examples before changing weights or presenting scores as predictive.

- Add tests for planner routing, score calculations, critic edge cases, and tracker persistence as the project grows.
- Consider a graph-based orchestrator only if the workflow needs branching, retries, checkpoints, or independently resumable steps; keep the existing deterministic behavior as a baseline.
- Add opt-in reminders only after a clear consent and notification design is defined.

9. Setup and Project Files

From the project root, install dependencies and start the Streamlit app:

```
pip install -r requirements.txt
streamlit run app.py
```

File or folder	Role
app.py	Streamlit page orchestration, session state, forms, and workflows
opportunityos/orchestrator.py	Sequential workflow coordination and hand-offs
opportunityos/agents/	Planner, Discovery, Matching, Follow-up, Critic, and Tracker components
opportunityos/resume/	Resume parsing, gap analysis, clarification, tailoring, truth checks, and exports
data/listings.csv	Synthetic listing dataset
data/tracker.db	Local SQLite application tracker; can be reseeded or cleared using demo controls
README.md and requirements.txt	Project overview, rubric notes, setup, and Python dependencies

9. References

1. Wanberg, C. R., Ali, A. A., & Csillag, B. (2020). Job seeking: The process and experience of looking for a job. *Annual Review of Organizational Psychology and Organizational Behavior, 7*, 315–337. https://doi.org/10.1146/annurev-orgpsych-012119-044939
   Cite this in the **problem background** when discussing job searching as a process involving multiple activities and decisions.
2. Wooldridge, M., & Jennings, N. R. (1995). Intelligent agents: Theory and practice. *The Knowledge Engineering Review, 10*(2), 115–152. https://doi.org/10.1017/S0269888900008122
   Cite this in the **agent architecture** section for foundational agent concepts. Describe Omnitrix accurately as a workflow of deterministic Python components; this reference does not mean the project uses autonomous LLM agents.
3. Streamlit, Inc. (n.d.). st.form. *Streamlit Documentation*.
   Cite this where you explain how the app groups inputs and runs the search workflow on submission.
4. Streamlit, Inc. (n.d.). Caching and state. *Streamlit Documentation*.
   Cite this in the implementation section for Streamlit caching and session-state concepts.
5. Python Software Foundation. (n.d.). sqlite3 — DB-API 2.0 interface for SQLite databases. *Python 3.13 Documentation*.
   Cite this in the **technical implementation** section if the app uses SQLite for tracker data.
6. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*. https://arxiv.org/abs/2210.03629